Tech Quest – Project Documentation

Tech Quest is a comprehensive learning and career-roadmap platform designed to help users explore different technology roles, track their progress, research topics, and build professional resumes. The platform is built with a modern frontend–backend architecture using RESTful API communication.

Project Overview

The project consists of two major components:

- **Backend API Server** – Handles authentication, progress tracking, feedback storage, and business logic.
- **Frontend Application** – A fully interactive UI that manages state, routing, progress tracking, PSE search, and the resume builder.

Backend

The backend is built using **Node.js**, **Express.js**, and **MongoDB** for persistent data storage.

Key Features & Technologies

- **Express Server**
  Provides RESTful API endpoints grouped by resources.
- **MongoDB (Mongoose)**
  Stores user accounts, progress, feedback, and other app data.
- **CORS**
  Allows secure communication between backend and frontend (domains: `https://techquest.pages.dev` and `http://localhost:3000`).
- **Rate Limiting**
  Protects the server from excessive or abusive requests.

Authentication

- Secure **user signup & login**.
- Passwords hashed using **bcrypt**.
- Stateless authentication using **JWT (JSON Web Tokens)**.
- Token validation for all protected routes.

API Endpoints

- POST /api/auth/signup – Register a new user
- POST /api/auth/login – Login and receive JWT
- GET /api/progress/:userId – Fetch user progress
- POST /api/progress – Update progress
- POST /api/feedback – Submit feedback
- GET /api/feedback – Retrieve feedback entries

Backend Notes

- Runs on **port 5000**
- Connects to MongoDB using **environment variables**

Frontend

The frontend is built using **React.js** with functional components and hooks.

Core Features

- **React Context API (AuthContext)**
  Manages authentication state, user details, tokens, and guest login.
- **Routing (react-router-dom)**
  Supports multi-page navigation with a clean SPA structure.
- **REST API Communication**
  Uses fetch() to send requests and receive user progress, feedback, and authentication data.

Authentication Flow

1. User logs in → backend returns JWT
2. JWT and user info stored in localStorage
3. AuthContext exposes login, logout, and guest options
4. Protected routes use JWT for access
5. UI updates reactively based on authentication state

State Management

- **useState** – local state
- **useEffect** – async calls, side effects, data fetching
- **Context API** – global authentication & app-wide data

Frontend–Backend Interaction

- Communication is **asynchronous** via RESTful API calls
- JWT tokens are sent in request headers
- Backend validates tokens before processing
- Responses are returned as JSON
- React updates UI instantly based on responses
- CORS ensures only trusted frontends can access the backend

Design Patterns & Architecture

Backend

- **RESTful API Structure**
  Clean separation of routes, controllers, and models.
- **Middleware Pattern**
  Handles CORS, rate limits, JSON parsing, validation, and error handling.
- **Token-Based Authentication (JWT)**
  Stateless and secure.
- **Input Validation**
  Ensures safe and predictable data handling.

Frontend

- **Component-Based Architecture**
  Small reusable React components.
- **Context API for Global State**
  Central auth management.
- **React Hooks (useState / useEffect)**
  Reactive UI updates.
- **React Router**
  SPA navigation system.

Interaction Flow

1. User performs an action in the frontend (login / progress).
2. Frontend authenticates and stores JWT.
3. Authenticated requests are sent to backend APIs.
4. Backend validates the token & processes logic.
5. MongoDB is queried or updated.
6. Backend returns JSON response.
7. Frontend updates UI reactively.

Resume Builder Feature

The Resume Builder is an integrated React component that helps users create professional resumes quickly.

Key Functionalities

- Forms to enter:
  - Basic info
  - Skills
  - Education
  - Projects
  - Experience
- Choose **5 customizable templates**
- Live preview in a new window
- One-click **PDF download**

Tech Stack for Resume Builder

- **React.js (v19.2.0)**
- **html2pdf.js** for PDF generation
- **ReactDOM / ReactDOMServer** for HTML string rendering

This feature provides an easy, beginner-friendly resume generation experience.

Conclusion

Tech Quest follows a clean separation of concerns between the frontend and backend. It uses modern development patterns such as RESTful architecture, JWT authentication, React Context API, hooks, and MongoDB to deliver a scalable and user-friendly learning platform.

Setup & Running

Backend

1. `npm install`
2. Configure environment variables (Mongo URI, JWT secret)
3. `node backend/server.js`

Frontend

1. npm install
2. npm start

Contact

For more information:
Email: mailto:karthikvelu0415@gamil.com